

routes\learn_routes.py

```
"""
Learning routes module - Optimized for Render's free tier (512MB) environment
"""
# Flask blueprint system for modular route organization
from flask import Blueprint, render_template, redirect, url_for, request
# Date/time handling for current timestamps and timezone awareness
from datetime import datetime, timezone
# Database and caching
from database.db_manager import db_manager
# CVE data retrieval service for educational content
from services.data.api_client import api_client
# CWE analysis and vendor risk assessment services
from services.analysis.cwe_processor import cwe_processor, get_vendor_risk_analysis
# Cache manager for optimizing memory usage
from services.cache.cache_manager import cache_manager
# Import config for lazy loading settings
import config
import gc

# Create learning blueprint with name 'learn' for route registration
learn_bp = Blueprint('learn', __name__)

# Static CWE titles for educational content - curated for learning value
CWE_TITLES = {
    "CWE-79": "Cross-Site Scripting",
    "CWE-89": "SQL Injection",
    "CWE-20": "Improper Input Validation",
    "CWE-22": "Path Traversal",
    "CWE-119": "Buffer Overflow",
    "CWE-200": "Information Exposure",
    "CWE-287": "Improper Authentication",
    "CWE-78": "OS Command Injection"
}

@learn_bp.route("/")
def learn():
    """Redirect to main learn topic"""
    # Check memory before handling request
    cache_manager.check_memory_usage()

    # Redirect root learning requests to default educational topic
    return redirect(url_for('learn.learn_topic', topic='what-is-cve'))

@learn_bp.route("/<topic>")
def learn_topic(topic):
    """Learn topic pages - OPTIMIZED FOR MEMORY USAGE"""
    # Check memory before handling request
    cache_manager.check_memory_usage()

    # Whitelist of valid educational topics for security and content control
    valid_topics = [
        'what-is-cve', 'what-is-cve', 'cvss-scores',
        'what-is-nvd-mitre', 'cve-vs-cve-vs-cvss'
    ]

    # Validate requested topic and redirect to default if invalid
    if topic not in valid_topics:
        return redirect(url_for('learn.learn_topic', topic='what-is-cve'))

    try:
        # Lazily load data based on config setting
        if config.LOAD_ALL_DATA_FOR_LEARN_PAGES:
            # FULL DATA LOADING - can cause memory issues on Render free tier
            # Get recent CVE data for educational examples (limit to 25 for performance)
            latest_cves = api_client.get_cves_last_30_days()[:25]

        # Enhanced data loading for CWE-focused educational content
        cwe_dict = {}
        cwe_severity = {}
        key_cwes = []
        key_cwe_titles = {}
```

```

if topic == 'what-is-cwe':
# Load advanced CWE analysis data using processor
cwe_severity = cwe_processor.get_cwe_severity_data(10)
cwe_dict = cwe_processor.get_cwe_details()

# Get processor's curated CWE list instead of static list
key_cwes = cwe_processor.get_key_cwes()
key_cwe_titles = cwe_processor.get_key_cwe_titles()

# Attempt to use advanced vendor risk analysis for educational insights
try:
vendor_risk_data = get_vendor_risk_analysis()

# Check if vendor risk data is available and properly formatted
if vendor_risk_data and 'top_10_cwes' in vendor_risk_data:
# Transform vendor risk data to chart-compatible format
top_10_data = vendor_risk_data['top_10_cwes']
severity_matrix = vendor_risk_data.get('severity_matrix', {})

# Initialize severity data structure for charts
cwe_severity = {
'labels': top_10_data.get('labels', []),
'indices': top_10_data.get('indices', []),
'data': {
'CRITICAL': [],
'HIGH': [],
'MEDIUM': [],
'LOW': [],
'UNKNOWN': []
}
}

# Populate severity data for each CWE from vendor analysis
for cwe_code in top_10_data.get('indices', []):
cwe_severities = severity_matrix.get(cwe_code, {})
cwe_severity['data']['CRITICAL'].append(cwe_severities.get('CRITICAL', 0))
cwe_severity['data']['HIGH'].append(cwe_severities.get('HIGH', 0))
cwe_severity['data']['MEDIUM'].append(cwe_severities.get('MEDIUM', 0))
cwe_severity['data']['LOW'].append(cwe_severities.get('LOW', 0))
cwe_severity['data']['UNKNOWN'].append(cwe_severities.get('UNKNOWN', 0))
except Exception as e:
# Log vendor risk analysis errors but continue with fallback data
print(f"[Learn] Error getting vendor risk data: {e}")
pass
else:
# Basic data loading for non-CWE topics
# Import severity analyzer locally to avoid circular imports
from services.analysis.severity_analyzer import get_severity_card_counts
cwe_severity = get_severity_card_counts()

# Use static CWE data for basic educational content
key_cwes = list(CWE_TITLES.keys())
key_cwe_titles = CWE_TITLES
else:
# MEMORY-EFFICIENT DATA LOADING FOR RENDER FREE TIER
# Use minimal data for educational pages

# Get only basic CVE data for examples - from database if possible
if db_manager.use_database:
# Get a small sample of recent CVEs from the database
latest_cves = db_manager.get_recent_cves(days=30, limit=25)
else:
# Fall back to API but limit to only 25 CVEs
latest_cves = api_client.get_cves_last_30_days()[:25]

# Use pre-computed or static data for the rest
cwe_dict = {}
key_cwes = list(CWE_TITLES.keys())
key_cwe_titles = CWE_TITLES

# Try to get cwe_severity from database cache
if db_manager.use_database:
severity_stats = db_manager.get_summary_stats('cwe_severity_data')
if severity_stats and 'data' in severity_stats:
cwe_severity = severity_stats['data']
else:
# Fall back to basic data

```

```

from services.analysis.severity_analyzer import get_severity_card_counts
cwe_severity = get_severity_card_counts()
else:
# Fall back to basic data
from services.analysis.severity_analyzer import get_severity_card_counts
cwe_severity = get_severity_card_counts()

# Render topic-specific template with aggregated educational data
result = render_template(
    f"learn/{topic}.html",
    cwe_dict=cwe_dict,
    cwe_severity=cwe_severity,
    latest_cves=latest_cves,
    key_cwes=key_cwes,
    key_cwe_titles=key_cwe_titles,
    now=datetime.now(timezone.utc), # Current timestamp for templates
    memory_optimized=not config.LOAD_ALL_DATA_FOR_LEARN_PAGES
)

# Clear variables to free memory
latest_cves = None
cwe_dict = None
cwe_severity = None

# Force garbage collection
gc.collect()

return result
except Exception as e:
# Log educational page errors for debugging
print(f"[Flask] Learn page error: {e}")
import traceback
traceback.print_exc()

# Provide safe fallback data to ensure educational continuity
return render_template(
    f"learn/{topic}.html",
    cwe_dict={},
    cwe_severity={},
    latest_cves=[],
    key_cwes=list(CWE_TITLES.keys()), # Use static CWE list as fallback
    key_cwe_titles=CWE_TITLES, # Use static titles as fallback
    now=datetime.now(timezone.utc),
    memory_optimized=True # Indicate memory optimization is active
)

```